

# THE OPEN SOURCE AUDIT

A Capstone Project for OSS NGMC Course

**Software Focus: LibreOffice**

---

|                            |                                      |
|----------------------------|--------------------------------------|
| <b>Student Name</b>        | Yash                                 |
| <b>Registration Number</b> | 24BCE10442                           |
| <b>Slot</b>                | A13                                  |
| <b>Date of Submission</b>  | 30 <sup>th</sup> March 2026          |
| <b>Chosen Software</b>     | LibreOffice                          |
| <b>License</b>             | Mozilla Public License 2.0 (MPL 2.0) |

## PART A — ORIGIN AND PHILOSOPHY (UNITS 1 & 2)

# A1. Origin Story — The Problem LibreOffice Was Created to Solve

To understand why LibreOffice exists, we must go back to the early 2000s, when the world of office productivity software was dominated almost entirely by Microsoft. For the average user or organisation, there was simply no viable alternative to Microsoft Office — you paid the licence fee, accepted the terms, and had no way to change, inspect, or share the software. This was not merely a financial burden; it represented a philosophical problem for those who believed software should be free and open.

In 2000, Sun Microsystems released OpenOffice.org as open-source software, making its codebase publicly available. For years, OpenOffice.org thrived as a community alternative to Microsoft Office. Developers from across the world contributed patches, translations, and improvements. Governments and educational institutions in Europe, India, and South America adopted it as a way to reduce dependence on proprietary software. It seemed like a genuine success story for the open-source movement.

Then, in 2010, Oracle Corporation acquired Sun Microsystems. Almost immediately, the open-source community noticed a change in how Oracle handled OpenOffice.org. Contributions from external developers were ignored or delayed. The governance structure became opaque. Oracle, a company primarily known for its enterprise database products and aggressive licensing policies, showed little genuine commitment to the community-driven philosophy that had made OpenOffice.org successful.

A group of core contributors, including key figures such as Michael Meeks, Bjoern Michaelson, Caolan McNamara, and others, made a decision that illustrates one of the most powerful aspects of open-source licensing: they forked the project. Because the OpenOffice.org codebase was available under a licence that permitted this, they took the code, created a new project called LibreOffice, and founded a new independent non-profit governance body called The Document Foundation in September 2010.

The name 'LibreOffice' was chosen deliberately — 'libre' from the Latin for 'free', not in the sense of price, but in the sense of freedom. The founders wanted a name that communicated their values from the very first word. The Document Foundation's founding manifesto stated that the new project would be governed transparently, would accept contributions from all developers regardless of employer, and would never be sold to a corporation that did not share its values.

Oracle eventually donated the OpenOffice.org trademark and code to the Apache Software Foundation, but by that point the community had largely moved to LibreOffice. The lesson of LibreOffice's origin is not just a business story — it is a demonstration that open-source licensing creates an insurance policy against corporate neglect. When a company stops caring for software, the community can take it back.

## A2. The Licence — What It Actually Says

### The Four Freedoms of Free Software

Richard Stallman of the Free Software Foundation defined the four essential freedoms that make software truly free:

- Freedom 0: The freedom to run the program for any purpose, without restriction.
- Freedom 1: The freedom to study how the program works and adapt it to your needs. Access to the source code is a prerequisite.
- Freedom 2: The freedom to redistribute copies so you can help others.
- Freedom 3: The freedom to improve the program and release your improvements to the public, so the whole community benefits.

LibreOffice is licensed under the Mozilla Public License 2.0 (MPL 2.0), and it grants all four of these freedoms. Anyone can download, use, modify, and redistribute LibreOffice. The source code is available on GitHub. The MPL 2.0 is a file-level copyleft licence, meaning that any modifications to MPL-licensed files must themselves be released under MPL 2.0, but larger works that merely use LibreOffice code are not required to be open source.

### Can a Company Modify and Sell It Without Sharing Changes?

This question is nuanced. Under MPL 2.0, if a company modifies LibreOffice source files, they must make those modifications available under MPL 2.0. However, they are free to build proprietary components on top of LibreOffice, as long as those components are kept in separate files. In practice, several commercial vendors, including Collabora and SUSE, offer enterprise versions of LibreOffice with proprietary extensions while contributing their core changes back to the community.

### GPL vs MIT: A Comparison

The GPL (GNU General Public License) is a strong copyleft licence. Any software that incorporates GPL code must also be released under the GPL. This creates a viral effect that ensures derived work remains free. The Linux kernel uses GPL v2.

The MIT licence is a permissive licence with almost no conditions. Anyone can take MIT code, modify it, incorporate it into proprietary software, and sell it without releasing their changes. This encourages maximum adoption but provides no guarantee that improvements flow back to the community.

If I were building software intended for maximum reuse in the enterprise world, I would choose MIT. If I were building something where I wanted to ensure the software remained free in perpetuity — that it could not be taken, locked down, and sold — I would choose GPL. The MPL 2.0 sits between

these: it protects the specific files that make up the project while being compatible with larger proprietary works.

### **Free as in Free Beer vs Free as in Freedom**

'Free as in free beer' means the software costs nothing to obtain. 'Free as in freedom' means you have the right to use, study, modify, and share the software. These are completely different things. A piece of software can be free to download but still restrict your freedoms — many mobile apps are free to install but prohibit reverse engineering, modification, or redistribution.

LibreOffice is both: it costs nothing, and it gives you full freedom. In contrast, Microsoft's online Office 365 tools are sometimes made available at no cost to students (free as in beer), but the source code is closed, the software cannot be modified, and users have no freedom over how it runs (not free as in freedom).

## **A3. The Ethics of Open Source**

### **Should All Software Be Open Source?**

The argument in favour is compelling: software is a form of knowledge, and knowledge grows more powerful when shared. The internet itself was built on open standards and open software. Open source allows independent security audits, prevents vendor lock-in, enables customisation, and creates a commons from which everyone benefits. In domains where public trust is essential — voting systems, medical devices, government services — the argument for open source becomes nearly irrefutable.

However, the argument against mandatory open-source is also real. Not all software is infrastructure. Some software represents years of private investment, research, and risk. A small startup building an innovative product cannot give it away and also pay its engineers. Open source thrives on volunteer effort and corporate sponsorship, but these are not always available. Forcing all software to be open could actually reduce the diversity and volume of software created, particularly in specialised or high-risk domains.

My position is that certain categories of software — especially that which runs public infrastructure, healthcare systems, or government services — should be open by default. Commercial software in competitive markets can remain proprietary, but those companies should contribute proportionally to the open-source ecosystem they depend on.

### **Is It Ethical for Companies to Profit from Free Community Labour?**

This is perhaps the most contested ethical question in the open-source world. Red Hat built a billion-dollar business on Linux. Google's data centres run on open-source software. Meta publishes

open-source frameworks but uses them to build highly profitable proprietary advertising platforms. Is this exploitation?

The honest answer is: sometimes yes, sometimes no. Red Hat contributes massively back to the Linux kernel and other projects — it is genuinely a partner to the open-source ecosystem. On the other hand, some companies take far more than they give. The open-source community has developed concepts like the 'tragedy of the commons' to describe this — shared resources can be depleted if individuals or corporations do not contribute back.

LibreOffice's story is instructive: Oracle took and neglected. The Document Foundation model — a non-profit with transparent governance and commercial partners who contribute rather than merely consume — is the ethical template for how companies should engage with open source.

## **Standing on the Shoulders of Giants**

Every piece of software built today uses tools, libraries, languages, and compilers created by people who came before and chose to share their work. When I write a Python script and run it on Linux, I am using the work of thousands of contributors. The phrase 'standing on the shoulders of giants' — originally about scientific knowledge — applies perfectly to software.

Open source does not undermine original innovation. It accelerates it. LibreOffice's developers did not need to reinvent text rendering or file format parsing from scratch — they built on existing open foundations. This is the network effect of open knowledge: every contribution makes the next contribution easier and more powerful.

## PART B — LINUX FOOTPRINT (UNIT 2)

# B. Linux Footprint of LibreOffice

### Installation Method

LibreOffice is available through most major Linux package managers. On Fedora and RHEL-based systems, it can be installed via DNF:

```
sudo dnf install libreoffice
```

On Debian and Ubuntu-based systems:

```
sudo apt install libreoffice
```

It is also available as a Flatpak (sandboxed universal package) and as a direct download from [libreoffice.org](https://libreoffice.org) for those who want the latest version regardless of their distribution's repository age.

### Key Directories

Once installed on a Fedora system, LibreOffice occupies several standard Linux filesystem locations:

- `/usr/bin/libreoffice` — the main executable wrapper script
- `/usr/lib64/libreoffice/` — core application binaries, shared libraries, and resources
- `/usr/share/libreoffice/` — templates, icons, language packs, and documentation
- `/etc/libreoffice/` — system-wide configuration files
- `~/.config/libreoffice/` — per-user configuration, macros, and customisations
- `/var/log/` — system logs capturing any install/update events

### User and Group

LibreOffice runs under the same Linux user account as the person who launches it — it does not run as a privileged system user. This is by design: office software should have no more access than the user themselves. If I run LibreOffice as user 'student', it reads and writes files with the permissions of that account. This is an important security principle: the principle of least privilege, which ensures that if LibreOffice is ever compromised by a malicious document, the damage is limited to that user's files and not the entire system.

### Service Management

LibreOffice is a desktop application, not a background service, so it does not have a systemd service unit for most desktop uses. However, LibreOffice can run in headless server mode for batch document conversion:

```
libreoffice --headless --convert-to pdf *.docx
```

In enterprise deployments with LibreOffice Online (Collabora), it can be managed as a containerised service via Docker or systemd:

```
sudo systemctl start collabora-online
```

## Update Model

On Fedora, updates to LibreOffice flow through the official RPM repositories maintained by the Fedora Project. The Document Foundation releases new versions, they are packaged by Fedora maintainers, and they arrive on user machines through:

```
sudo dnf upgrade libreoffice
```

Security patches are prioritised and pushed as soon as they are available. The transparent nature of open-source development means that vulnerabilities are often identified and reported by community members and fixed faster than in proprietary software where external audit is impossible.

## PART C — THE FOSS ECOSYSTEM (UNITS 3 & 4)

# C. The FOSS Ecosystem Around LibreOffice

### Dependencies and Libraries

LibreOffice is one of the most complex open-source applications in existence, consisting of millions of lines of code and depending on hundreds of external libraries. Key dependencies include:

- Cairo and Skia — graphics rendering libraries used for drawing document content
- ICU (International Components for Unicode) — handles text in dozens of scripts and languages
- Boost — a set of C++ libraries used extensively throughout the codebase
- OpenSSL / NSS — cryptographic libraries for secure document handling and macro signing
- Hunspell — the spell-checking engine also used by Firefox and several other applications
- CUPS — the Common Unix Printing System for print integration
- fontconfig and FreeType — font discovery and rendering on Linux
- Python — an embedded Python runtime enabling the macro scripting engine

### What LibreOffice Has Enabled and Inspired

LibreOffice has had a profound influence on the open-source software landscape. Its most direct offspring is Collabora Online, a cloud-hosted version of LibreOffice built by Collabora Productivity that competes directly with Google Docs and Microsoft Office 365 Online. Collabora is the engine behind Nextcloud Office, allowing organisations to run their own private, fully open-source cloud office suite.

The ODF (Open Document Format) standard, championed by LibreOffice and The Document Foundation, has been adopted by governments worldwide as a mandatory format for public documents. Germany, France, and the United Kingdom have all issued guidance mandating ODF for public sector documents. This is a direct policy consequence of LibreOffice's advocacy.

### Community and Governance

LibreOffice is governed by The Document Foundation (TDF), a German non-profit foundation (Stiftung). The Board of Directors is elected by community members. Key developer decisions are made through public mailing lists, and anyone can review meeting minutes, financial reports, and roadmap discussions.

The community communicates through:

- Mailing lists at [lists.freedesktop.org](https://lists.freedesktop.org) and [tdf.io](https://tdf.io)
- The #libreoffice IRC channel on Libera.Chat
- The LibreOffice Community blog and Planet
- Annual LibreOffice Conference (LibOCon), held in a different city each year



- Bugzilla at [bugs.documentfoundation.org](https://bugs.documentfoundation.org) for issue tracking

## **Relation to the LAMP Stack**

LibreOffice is not part of the classic LAMP stack (Linux, Apache, MySQL, PHP), but it intersects with web infrastructure in important ways. Collabora Online can be deployed behind an Apache or Nginx reverse proxy, integrating LibreOffice's editing capabilities into web applications. LAMP-based content management systems like Moodle and Nextcloud use LibreOffice's headless mode to convert uploaded documents to PDF for browser-based preview.

## PART D — OPEN SOURCE VS PROPRIETARY (CRITICAL ANALYSIS)

## D. Open Source vs Proprietary: LibreOffice vs Microsoft Office

The following table provides a structured comparison across the key dimensions discussed in the course:

| Dimension                        | LibreOffice (Open Source)                                                                                                      | Microsoft Office (Proprietary)                                                                                        |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Cost</b>                      | Completely free to download, use, and distribute. No licensing fees for individuals or organisations.                          | Requires a paid subscription (Microsoft 365) or one-time purchase. Costs can range from \$70 to \$400+ per user.      |
| <b>Security (code audit)</b>     | Full source code is publicly available. Any security researcher or developer worldwide can audit and report vulnerabilities.   | Source code is closed. Security relies entirely on Microsoft's internal team. External audit is impossible.           |
| <b>Support &amp; Reliability</b> | Community forums, The Document Foundation support, and optional paid support from vendors. Highly reliable for most workflows. | 24/7 official Microsoft support, enterprise SLAs, extensive documentation. Preferred in corporate environments.       |
| <b>Freedom to Modify</b>         | Users can modify, adapt, and redistribute under MPL 2.0. Organisations can customise for their specific needs.                 | No modification allowed. Users must accept the software as Microsoft provides it.                                     |
| <b>Community vs Corporate</b>    | Governed by The Document Foundation, a non-profit. Decisions are community-driven with transparent governance.                 | Controlled entirely by Microsoft. Product direction is driven by business priorities and shareholder interests.       |
| <b>Your Overall Verdict</b>      | Best for individuals, schools, governments, and organisations that value freedom, cost savings, and data sovereignty.          | Best for large enterprises with existing Microsoft ecosystems requiring deep integration with Teams, SharePoint, etc. |

### Conclusion and Recommendation

After conducting this audit, I would confidently recommend LibreOffice for deployment in educational institutions, government agencies, NGOs, and small-to-medium businesses that prioritise cost efficiency and digital sovereignty. The software is mature, feature-complete, and actively maintained. For an institution in India, where software licensing costs are a significant burden and where government policy increasingly favours open standards, LibreOffice represents both a practical and a principled choice.

Would I contribute to it? Yes. LibreOffice has a well-documented 'Easy Hacks' list on its wiki, specifically maintained to help new contributors begin with manageable tasks. The codebase is in C++ and Python, and the project has explicit processes for mentoring student contributors through

Google Summer of Code. The combination of real-world impact, active mentorship, and transparent governance makes it one of the most welcoming open-source projects for student contributors.

## SHELL SCRIPTS — PRACTICAL LINUX DEMONSTRATIONS (UNIT 2 &amp; 5)

## Script 1 — System Identity Report

|          |                                                                     |
|----------|---------------------------------------------------------------------|
| Script   | Script 1: System Identity Report                                    |
| Concepts | Variables, echo, command substitution \$(), basic output formatting |

### Code

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Yash | Course: Open Source Software

# --- Variables ---
STUDENT_NAME=" Yash "
SOFTWARE_CHOICE="LibreOffice"

# --- System Information ---
DISTRO=$(cat /etc/fedora-release)
KERNEL=$(uname -r)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
CURRENT_DATE=$(date)

# --- Display ---
echo "=====
echo " Open Source Audit - $STUDENT_NAME"
echo "=====
echo "Chosen Software : $SOFTWARE_CHOICE"
echo "Distribution      : $DISTRO"
echo "Kernel Version    : $KERNEL"
echo "Current User       : $USER_NAME"
echo "Home Directory     : $HOME_DIR"
echo "System Uptime      : $UPTIME"
echo "Date and Time      : $CURRENT_DATE"
echo ""
echo "License: Fedora Linux is distributed under"
echo "various free software licenses, mainly the GPL."
echo "=====
```

## Output

```
=====
Open Source Audit - Yash
Fedora Linux System Identity Report
=====
Chosen Software : LibreOffice
Distribution    : Fedora Linux 43 (Workstation Edition)
Kernel Version : 6.19.8-200.fc43.x86_64
Current User   : mitul
Home Directory : /home/mitul
System Uptime  : up 46 minutes
Current Date   : Mon Mar 30 07:43:43 PM IST 2026

Fedora Linux is licensed mainly under the GNU GPL
which guarantees users the freedom to study, modify
and share the operating system.
```

## Explanation

This script demonstrates fundamental shell scripting concepts. It begins by declaring string variables using the `NAME=value` syntax. It then uses command substitution — the `$()` operator — to run system commands (such as `uname -r`, `whoami`, `uptime -p`, and `date`) and capture their output into variables at runtime. The `echo` command is used throughout to format and display a structured welcome screen. This is the most basic and foundational pattern in shell scripting: run a command, capture its output, display it to the user.

The script reads the distribution name from `/etc/fedora-release` using the `cat` command, demonstrating how Linux exposes system information as plain text files. The `$HOME` variable is a pre-existing environment variable that every Linux shell session provides automatically. The formatted output uses `echo` with strings that combine literal text and variable references, producing a readable system identity card.

## Script 2 — FOSS Package Inspector

|                 |                                                       |
|-----------------|-------------------------------------------------------|
| <b>Script</b>   | Script 2: FOSS Package Inspector                      |
| <b>Concepts</b> | if-then-else, case statement, rpm -qi, pipe with grep |

### Code

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Yash

PACKAGE="libreoffice"

echo "=====
echo "          FOSS PACKAGE INSPECTOR"
echo "=====

if rpm -q $PACKAGE &>/dev/null; then
    echo "$PACKAGE is installed."
    echo ""
    rpm -qi $PACKAGE | grep -E 'Name|Version|Release|License|Summary'
else
    echo "$PACKAGE is NOT installed."
fi

echo ""
echo "PHILOSOPHY NOTE"
echo "-----"
case $PACKAGE in
    libreoffice)
        echo "LibreOffice: a community-driven office suite that protects user freedom."
        ;;
    firefox)
        echo "Firefox: a browser created to keep the web open and independent."
        ;;
    git)
        echo "Git: version control built to support open collaboration."
        ;;
    vlc)
        echo "VLC: media software that proves open source can be universal."
        ;;
    *)
        echo "Unknown package."
        ;;
esac
```

## Output

```
=====
FOSS PACKAGE INSPECTOR
=====
libreoffice is installed.

Name       : libreoffice
Version    : 25.8.5.2
Release    : 1.fc43
License     : MPL-2.0 AND Apache-2.0 AND LGPL-3.0-only AND LGPL-3.0-or-later AND
             CC0-1.0 AND BSD-3-Clause AND (LGPL-2.1-only OR SISSL) AND (MPL-2.0 OR LGPL-3.0-
             or-later) AND (MPL-2.0 OR LGPL-2.1-or-later) AND (MPL-1.1 OR GPL-2.0-only OR LGP
             L-2.1-only) AND MIT
Summary    : Free Software Productivity Suite

PHILOSOPHY NOTE
-----
LibreOffice: a community-driven office suite that protects user freedom.
```

## Explanation

This script demonstrates conditional logic and pattern matching. The if-then-else construct checks whether the LibreOffice package is installed by running `rpm -q` and redirecting all output to `/dev/null` — we only care about the exit code, not the output itself. If the package is found, `rpm -qi` retrieves detailed package metadata, which is then piped through `grep` with a regular expression to extract only the relevant fields (Name, Version, Release, License, Summary).

The case statement implements a multi-branch conditional that matches the package name against several known software values and prints a philosophy note for each. This mimics a pattern commonly used in system administration scripts that need to handle different software types in different ways. The `*)` wildcard at the end of the case serves as a default fallback, demonstrating good defensive scripting practice.

## Script 3 — Disk and Permission Auditor

|                 |                                        |
|-----------------|----------------------------------------|
| <b>Script</b>   | Script 3: Disk and Permission Auditor  |
| <b>Concepts</b> | for loop, arrays, du, ls -ld, awk, cut |

### Code

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Yash

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "====="
echo "      DISK AND PERMISSION AUDITOR"
echo "====="

for DIR in "${DIRS[@]}"
do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        echo "$DIR"
        echo "Permissions : $PERMS"
        echo "Size       : $SIZE"
        echo "-----"
    else
        echo "$DIR does not exist."
    fi
done

echo ""
echo "LibreOffice Configuration Directory"
echo "-----"
CONFIG="/etc/libreoffice"
if [ -d "$CONFIG" ]; then
    ls -ld "$CONFIG"
else
    echo "$CONFIG not found on this system."
fi
```



## Output

```
=====
      DISK AND PERMISSION AUDITOR
=====
/etc
Permissions : drwxr-xr-x. root root
Size       : 34M
-----
/var/log
Permissions : drwxr-xr-x. root root
Size       : 93M
-----
/home
Permissions : drwxr-xr-x. root root
Size       : 1.6G
-----
/usr/bin
Permissions : dr-xr-xr-x. root root
Size       : 439M
-----
/tmp
Permissions : drwxrwxrwt. root root
Size       : 8.0K
-----

LibreOffice Configuration Directory
-----
/etc/libreoffice not found on this system.
```

## Explanation

This script uses a Bash array to store a list of directory paths, then iterates over them using a for loop with the `${DIRS[@]}` expansion syntax, which expands every element of the array. For each directory, it first checks existence using `[ -d ]` before attempting to gather data — this prevents errors on systems where some directories may not exist.

The `ls -ld` command lists directory metadata rather than contents, and `awk '{print $1, $3, $4}'` extracts the permissions field, owner, and group from the output. The `du -sh` command reports disk usage in human-readable form, and `cut -f1` extracts just the size field, discarding the path that `du` appends. After the main loop, the script specifically checks for LibreOffice's configuration directory, tying the practical skill to the project under audit.

## Script 4 — Log File Analyzer

|                 |                                                                                            |
|-----------------|--------------------------------------------------------------------------------------------|
| <b>Script</b>   | Script 4: Log File Analyzer                                                                |
| <b>Concepts</b> | while read loop, if-then, counter variables, command-line arguments (\$1, \$2), grep, tail |

### Code

```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./log_analyzer.sh /var/log/messages [keyword]

LOGFILE=$1
KEYWORD=${2:-"error"}

echo "=====
echo "          LOG FILE ANALYZER"
echo "=====

if [ ! -f "$LOGFILE" ]; then
    echo "Error: File not found."
    exit 1
fi

COUNT=$(grep -i "$KEYWORD" "$LOGFILE" | wc -l)
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
echo ""
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

### Output

```
=====
LOG FILE ANALYZER
=====
Keyword 'error' found 561 times in /var/log/messages

Last 5 matching lines:
Mar 29 12:52:28 fedora google-chrome.desktop[4263]: [12458:12458:0329/125228.542
838:ERROR:base/memory/shared_memory_switch.cc:289] Failed global descriptor look
up: 7
Mar 29 12:52:42 fedora google-chrome.desktop[4263]: [12509:12509:0329/125242.971
270:ERROR:base/memory/shared_memory_switch.cc:289] Failed global descriptor look
up: 7
Mar 29 12:56:11 fedora gnome-shell[2802]: libinput error: event8 - ELAN0788:00
04F3:321A Touchpad: kernel bug: Touch jump detected and discarded.
Mar 29 12:57:35 fedora google-chrome.desktop[4263]: [4257:4295:0329/125735.05456
6:ERROR:google_apis/gcm/engine/connection_factory_impl.cc:483] ConnectionHandler
failed with net error: -2
Mar 29 13:03:49 fedora gnome-shell[2802]: libinput error: event8 - ELAN0788:00
04F3:321A Touchpad: kernel bug: Touch jump detected and discarded.
```

## Explanation

This script accepts command-line arguments using the positional parameters \$1 (the log file path) and \$2 (the keyword to search for). The \${2:-error} syntax demonstrates parameter substitution with a default value — if the user does not provide a second argument, it defaults to the string 'error'. This is a common pattern in production shell scripts that makes them flexible while remaining safe to use without all arguments.

The [ ! -f ] test checks that the file exists and is a regular file before proceeding, exiting cleanly with a non-zero exit code if not. Rather than a slow while-read loop, this implementation uses grep -i (case-insensitive) piped to wc -l for an efficient count, which is significantly faster on large log files. The tail -5 at the end shows only the five most recent matching lines, which is the most operationally useful information from a log analysis.

## Script 5 — The Open Source Manifesto Generator

|          |                                                                                 |
|----------|---------------------------------------------------------------------------------|
| Script   | Script 5: Open Source Manifesto Generator                                       |
| Concepts | read for user input, string variables, file writing with > and >>, date command |

### Code

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Yash

echo "Answer three questions to generate your manifesto."
echo ""
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does freedom mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

echo "Open Source Manifesto" > $OUTPUT
echo "Generated on: $DATE" >> $OUTPUT
echo "" >> $OUTPUT
echo "I use $TOOL every day because open-source software gives me $FREEDOM." >> $OUTPUT
echo "If I could build something and share it freely, I would create $BUILD." >> $OUTPUT
echo "I believe knowledge grows when people collaborate openly and help each other." >> $OUTPUT

echo "Manifesto saved to $OUTPUT"
echo ""
cat $OUTPUT
```

### Output

```
Answer three questions to generate your manifesto.

1. Name one open-source tool you use every day: LibreOffice
2. In one word, what does freedom mean to you? liberty
3. Name one thing you would build and share freely: Document editor

Manifesto saved to manifesto_mitul.txt

Open Source Manifesto
Generated on: 29 March 2026

I use LibreOffice every day because open-source software gives me liberty.
If I could build something and share it freely, I would create Document editor.
I believe knowledge grows when people collaborate openly and help each other.
```

## Explanation

This script demonstrates interactive shell scripting using the `read -p` command, which displays a prompt and waits for user input, storing the result in a named variable. Three questions are asked sequentially, and the answers are stored in `TOOL`, `FREEDOM`, and `BUILD`. The `date` command with a custom format string  `+%d %B %Y` produces a human-readable date.

File creation uses the `>` operator to create a new file (or overwrite an existing one) with the first `echo`, and the `>>` operator for all subsequent lines, which appends to the file. The output filename is dynamically generated using `$(whoami)` so that each user who runs the script gets their own manifesto file. The script ends with `cat $OUTPUT` to immediately display what was written, providing instant feedback to the user. This combination of interactive input, string composition, and file writing is a core skill for shell automation.

# References and Resources

## Primary Sources

- The Document Foundation — [libreoffice.org](https://libreoffice.org) and [documentfoundation.org](https://documentfoundation.org)
- LibreOffice source code repository — [github.com/LibreOffice](https://github.com/LibreOffice)
- Mozilla Public License 2.0 full text — [mozilla.org/en-US/MPL/2.0/](https://mozilla.org/en-US/MPL/2.0/)
- GNU Free Software Definition — [gnu.org/philosophy/free-sw.html](https://gnu.org/philosophy/free-sw.html)
- Open Source Initiative OSD — [opensource.org/osd](https://opensource.org/osd)

## Philosophy and History

- Eric S. Raymond — The Cathedral and the Bazaar ([catb.org/~esr/writings/cathedral-bazaar](https://catb.org/~esr/writings/cathedral-bazaar))
- Richard Stallman — Free Software, Free Society (GNU Press, 2002)
- The Document Foundation founding announcement, September 2010

## Technical References

- The Linux Command Line by William Shotts — [linuxcommand.org](https://linuxcommand.org)
- GNU Bash Manual — [gnu.org/software/bash/manual](https://gnu.org/software/bash/manual)
- Fedora Linux documentation — [docs.fedoraproject.org](https://docs.fedoraproject.org)
- SPDX License List — [spdx.org/licenses](https://spdx.org/licenses)